

CUESTIONARIO 2: Operators

1. Which of the following Java operators can be used with boolean variables? (Choose all that apply.)

A. `==`

B. `+`

C. `--`

D. `!`

E. `%`

F. `~`

G. Cast with `(boolean)`

Mi justificación

Con booleanos no puedes realizar operaciones aritméticas como suma, resta, multiplicación, división o residuo. Solo puedes realizar operaciones de comparación o lógicas. Por lo tanto A y D son válidas. G igual es válido pero sería algo raro castear (boolean) a otro booleano.

Respuesta: A,D,G

1. **A, D, G.** La opción A es el operador de igualdad y puede utilizarse con tipos primitivos y referencias a objetos. Las opciones B y C son operadores aritméticos y no pueden aplicarse a un valor boolean. La opción D es el operador de complemento lógico y se utiliza exclusivamente con valores boolean. La opción E es el operador módulo, que solo puede utilizarse con tipos numéricos primitivos. La opción F es el operador de complemento a nivel de bits y solo puede aplicarse a valores enteros. Finalmente, la opción G es correcta, ya que se puede hacer un cast de una variable boolean, porque boolean es un tipo.

2. What data type (or types) will allow the following code snippet to compile? (Choose all that apply.)

```
byte apples = 5;  
short oranges = 10;  
_____ bananas = apples + oranges;
```

- A. ☒ int
- B. ☒ long
- C. ☐ boolean
- D. ☒ double
- E. ☐ short
- F. ☐ byte

Mi justificación
La suma de short y byte da como resultado un int, así que A es válida. Ahora bien, la variable puede ser de una que haga casteo automático con int, así que aquellas variables que ocupan más espacio que int hacen casteo automático, en este caso son long y double. Entonces las correctas son A,B y D

2. A, B, D. La expresión `apples + oranges` se promociona automáticamente a `int`, por lo que funcionarán `int` y los tipos de datos a los que `int` puede convertirse automáticamente. Las opciones A, B y D son de este tipo. La opción C no funciona porque `boolean` no es un tipo numérico. Las opciones E y F no funcionan sin un cast explícito a un tipo de dato más pequeño.

3. What change, when applied independently, would allow the following code snippet to compile? (Choose all that apply.)

```
3: long ear = 10;  
4: int hearing = 2 * ear;
```

- A. No change; it compiles as is.
- B. Cast `ear` on line 4 to `int`.
- C. Change the data type of `ear` on line 3 to `short`.
- D. Cast `2 * ear` on line 4 to `int`.
- E. Change the data type of `hearing` on line 4 to `short`.
- F. Change the data type of `hearing` on line 4 to `long`.

Mi justificación

El programa no compila en su formato original porque `2 * ear` es `long` al ser `ear` un `long` y no tiene casteo automático `int = long`. Para solucionar esto se puede hacer que

- B. Se cambie `ear` de tipo `long` a `int` en la línea 3, así la 4 es `int = int`
- C. Cambiar el tipo de `ear` a `short` en la línea 3 pues `int = short` si tiene casteo automático en la línea 4
- D. Castear `2*ear` en la línea 4 a `int` pues ese valor es el `long` que hace que `int = long` no funcione.
- F. Cambiar el tipo de dato de `hearing` a `long`. Para así respetar el tipo `long = long`.

Respuesta: B,C,D,F

3. **B, C, D, F.** El código no compila tal como está, por lo que la opción A no es correcta. El valor `2 * ear` se promociona automáticamente a `long` y no puede almacenarse automáticamente en `hearing`, que es un `int`. Las opciones B, C y D solucionan este problema reduciendo el valor `long` a `int`. La opción E no soluciona el problema y, de hecho, lo empeora al intentar colocar el valor en un tipo de dato más pequeño. La opción F soluciona el problema aumentando el tipo de dato de la asignación para que se permita `long`.

4. What is the output of the following code snippet?

```
3: boolean canine = true, wolf = true;
4: int teeth = 20;
5: canine = (teeth != 10) ^ (wolf=false);
6: System.out.println(canine+", "+teeth+", "+wolf);
```

- A. true, 20, true
- B. true, 20, false**
- C. false, 10, true
- D. false, 20, false
- E. The code will not compile because of line 5.
- F. None of the above.

Mi justificación
En la línea 5 se hace lo siguiente canine = (true porque teeth no vale 10) XOR (Wolf cambia a falso) Canine = true XOR false = true (canine, teeth, Wolf)=(True, 20, false) Es la B

4. **B.** El código compila y se ejecuta sin problemas, por lo que la opción E no es correcta. Este ejemplo es engañoso debido al segundo operador de asignación incluido en la línea 5. La expresión (wolf=false) asigna el valor false a wolf y devuelve false para toda la expresión. Como teeth no es igual a 10, el lado izquierdo devuelve true; por lo tanto, el XOR exclusivo (^) de toda la expresión asignada a canine es true. La salida refleja estas asignaciones, sin cambios en teeth, por lo que B es la única respuesta correcta.

5. Which of the following operators are ranked in increasing or the same order of precedence? Assume the + operator is binary addition, not the unary form. (Choose all that apply.)

- A. +, *, %, --
- B. ++, (int), *
- C. =, ==, !
- D. (short), =, !, *
- E. *, /, %, +, ==
- F. !, |, |, &
- G. ^, +, =, +=

Mi justificación

La A es correcta por PEMDAS, es decir multiplicación y división tienen mayor jerarquía que suma. Y el - - se que se evalúa primero.
La G igual es correcta porque primero realizaría operaciones lógicas, luego aritméticas y luego asignación o comparación de valores.
La B y D no son porque el casteo tiene la precedencia más alta así que iría al final.

5. **A, C.** Las opciones A y C muestran operadores en orden de precedencia creciente o en el mismo nivel. Las opciones B y E están en orden de precedencia decreciente o en el mismo nivel. Las opciones D, F y G no están en un orden creciente ni decreciente. En D, el operador de asignación (=) está entre dos operadores unarios, mientras que el operador de multiplicación (*) está colocado incorrectamente en la posición de mayor precedencia. En F, el operador de complemento lógico (!) tiene la mayor precedencia, por lo que debería estar al final. En G, los operadores de asignación tienen la menor precedencia, no la mayor, por lo que los dos últimos operadores deberían estar primero.

6. What is the output of the following program?

```
1: public class CandyCounter {  
2:     static long addCandy(double fruit, float vegetables) {  
3:         return (int)fruit+vegetables;  
4:     }  
5:  
6:     public static void main(String[] args) {  
7:         System.out.print(addCandy(1.4, 2.4f) + ", ");  
8:         System.out.print(addCandy(1.9, (float)4) + ", ");  
9:         System.out.print(addCandy((long)(int)(short)2, (float)4)); } }
```

- A. 4, 6, 6.0
- B. 3, 5, 6**
- C. 3, 6, 6
- D. 4, 5, 6
- E. The code does not compile because of line 9.
- F. None of the above.

Mi justificación

No importa los números que metas como parámetros a addCandy, el resultado se castea a a int.
La primera invocación da 3.8, casteado a int es 3
La segunda invocación de 4.9, casteado a int es 4
La tercera invocación de 6.0, casteado a int es 6

6. F. El código no compila porque la línea 3 contiene un error de compilación. El cast (int) se aplica a fruit, no a la expresión fruit+vegetables. Como el operador de cast tiene mayor precedencia que el operador de suma, se aplica primero a fruit, pero la expresión completa se promociona a float debido a que vegetables es float. El resultado no puede devolverse como long en el método addCandy() sin un cast. Por esta razón, la opción F es correcta. Si se agregaran paréntesis alrededor de fruit+vegetables, la salida sería 3, 5, 6 y la opción B sería correcta. Recuerda que convertir números de punto flotante a valores enteros produce **truncamiento**, no redondeo.

7. What is the output of the following code snippet?

```
int ph = 7, vis = 2;  
boolean clear = vis > 1 & (vis < 9 || ph < 2);  
boolean safe = (vis > 2) && (ph++ > 1);  
boolean tasty = 7 <= --ph;  
System.out.println(clear + "-" + safe + "-" + tasty);
```

- A. true-true-true
- B. true-true-false
- C. true-false-true
- D. true-false-false
- E. false-true-true
- F. false-true-false
- G. false-false-true
- H. false-false-false

Mi justificación
Clear = true & (true OR false) = true & true = true Safe = false AND (8>1) = false AND true = false Tasty = 7 <= (8-1) = 7<= 7 = true Por ello la respuesta es C

7. D. En la primera expresión boolean, vis es 2 y ph es 7, por lo que la expresión se evalúa como true & (true || false), que se reduce a true. La segunda expresión boolean utiliza el operador condicional y, como (vis > 2) es false, el lado derecho no se evalúa, dejando ph en 7. En la última asignación, ph es 7 y el operador de pre-decremento se aplica primero, reduciendo la expresión a 7 <= 6, lo que produce una asignación de false. Por estas razones, la opción D es correcta.

8. What is the output of the following code snippet?

```
4: int pig = (short)4;  
5: pig = pig++;  
6: long goat = (int)2;  
7: goat -= 1.0;  
8: System.out.print(pig + " - " + goat);
```

- A. 4 - 1
- B. 4 - 2
- C. 5 - 1
- D. 5 - 2
- E. The code does not compile due to line 7.
- F. None of the above.

Mi justificación
El código no tiene errores de compilación porque en la línea 7 estoy tratando de hacer Goat = goat - 1.0 Goat = int - double Goat = double Goat = 1.0 Entonces lo que imprime es el valor de pig - goat, quedando 4 - 1

8. A. El código compila y se ejecuta sin problemas, por lo que la opción E es incorrecta. La línea 7 no produce un error de compilación porque el operador compuesto realiza el cast automáticamente. La línea 5 incrementa pig en 1, pero devuelve el valor original de 4 porque utiliza el operador de post-incremento. Después, pig recibe este valor y la operación de incremento se descarta. La línea 7 simplemente reduce el valor de goat en 1, dando como resultado 4 -1, por lo que la opción A es correcta.

9. What are the unique outputs of the following code snippet? (Choose all that apply.)

```
int a = 2, b = 4, c = 2;
System.out.println(a > 2 ? --c : b++);
System.out.println(b = (a!=c ? a : b++));
System.out.println(a > b ? b < c ? b : 2 : 1);
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6
- G. The code does not compile.

Mi justificación
En la primera línea $a > 2$ es false así que se imprime $b++$ pero primero se imprime 4 y luego $b = 4 + 1 = 5$, así que se imprime 4 En la segunda línea $a!=c$ es false, así que se ejecuta $b = b++$, pero ocurre algo similar a la primera línea y se imprime primero 5 y luego $b = 5 + 1 = 6$ En la tercera línea, si el primer condicional es false entonces imprime el último valor, y así fue ya que $a > b$ es false porque $2 > 6$ es false, así que imprime el 1. Resultado: A,D,E

9. **A, D, E.** El código compila sin problemas, por lo que la opción G es incorrecta. En la primera expresión, $a > 2$ es false, por lo que b se incrementa a 5; pero como se utiliza el operador de post-incremento, se imprime 4, haciendo que D sea correcta. El $--c$ no se aplica porque solo se evaluó una de las expresiones del lado derecho. En la segunda expresión, $a!=c$ es false porque c nunca fue modificado. Como b es 5 debido a la línea anterior y se utiliza el post-incremento, $b++$ devuelve 5. El resultado se asigna nuevamente a b mediante el operador de asignación, sobrescribiendo el valor incrementado de b y mostrando 5, haciendo que E sea correcta. En la última expresión no son necesarios los paréntesis, aunque su ausencia puede hacer que las expresiones ternarias sean difíciles de leer. De las líneas anteriores tenemos $a = 2$, $b = 5$ y $c = 2$. Podemos reescribir la expresión con paréntesis como $(2 > 5 ? (5 < 2 ? 5 : 2) : 1)$. La segunda expresión ternaria nunca se evalúa porque $2 > 5$ es false, y la expresión devuelve 1, haciendo que A sea correcta.

10. What are the unique outputs of the following code snippet? (Choose all that apply.)

```
short height = 1, weight = 3;
short zebra = (byte) weight * (byte) height;
double ox = 1 + height * 2 + weight;
long giraffe = 1 + 9 % height + 1;
System.out.println(zebra);
System.out.println(ox);
System.out.println(giraffe);
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6

G. The code does not compile.

Mi justificación
El código no compila porque en la definición de zebra están realizando la multiplicación de dos variables de tipo byte, lo cual da como resultado un int, y se quiere guardar en una variable short. Si se quiere guardar short = int se necesita un casteo explícito. Respuesta: G

10. G. El código no compila debido a un error en la segunda línea. Aunque tanto height como weight se convierten a byte, el operador de multiplicación los promociona automáticamente a int, lo que provoca un intento de almacenar un int en una variable short. Por esta razón, el código no compila y la opción G es la única correcta. Esta línea contiene el único error de compilación.

11. What is the output of the following code?

```
11: int sample1 = (2 * 4) % 3;  
12: int sample2 = 3 * 2 % 3;  
13: int sample3 = 5 * (1 % 2);  
14: System.out.println(sample1 + ", " + sample2 + ", " + sample3);
```

- A. 0, 0, 5
- B. 1, 2, 10
- C. 2, 1, 5
- D. 2, 0, 5**
- E. 3, 1, 10
- F. 3, 2, 6
- G. The code does not compile.

Mi justificación
Sample1 = $8\%3 = 2$ Sample2 = $(3*2)\%3 = 6\%3 = 0$ Sample3 = $5 * (1\%2) = 5*1 = 5$ Respuesta: D

11. D. * y % tienen la misma precedencia, por lo que la expresión se evalúa de izquierda a derecha, a menos que haya paréntesis. La primera expresión se evalúa como $8\%3$, cuyo residuo es 2. La segunda expresión se evalúa de izquierda a derecha, ya que * y % tienen la misma precedencia, y se reduce a $6\%3$, que es 0. La última expresión se reduce a $5 * 1$, que es 5. Por lo tanto, la salida en la línea 14 es 2, 0, 5, haciendo que D sea correcta.

12. The _____ operator increases a value and returns the original value, while the _____ operator decreases a value and returns the new value.
- A. post-increment, post-increment
 - B. pre-decrement, post-decrement
 - C. post-increment, post-decrement
 - D. post-increment, pre-decrement
 - E. pre-increment, pre-decrement
 - F. pre-increment, post-decrement

Mi justificación
Post = Regresa el valor antes de la operación Pre = Regresa el valor después de la operación Por esta lógica, es la D.

12. D. El prefijo indica que la operación se aplica primero y después se devuelve el nuevo valor, mientras que el postfijo indica que primero se devuelve el valor original y después se aplica la operación. Incrementar aumenta el valor, mientras que decrementar lo disminuye. Por estas razones, la opción D es correcta.

13. What is the output of the following code snippet?

```
boolean sunny = true, raining = false, sunday = true;
boolean goingToTheStore = sunny & raining ^ sunday;
boolean goingToTheZoo = sunday && !raining;
boolean stayingHome = !(goingToTheStore && goingToTheZoo);
System.out.println(goingToTheStore + "-" + goingToTheZoo
    + "-" +stayingHome);
```

- A. true-false-false
- B. false-true-false
- C. true-true-true
- D. false-true-true
- E. false-false-false
- F. true-true-false**
- G. None of the above

Mi justificación
goingToTheStore = true AND (true XOR false) = true AND true = true goingToTheZoo = true AND NOT(false) = true AND true = true stayingHome = NOT(true and true) = NOT(true) = false Respuesta: F

13. F. La primera expresión se evalúa de izquierda a derecha porque los operadores & y ^ tienen la misma precedencia, por lo que podemos reducirla a $\text{false} \wedge \text{sunday}$, que es true porque sunday es true. En la segunda expresión, aplicamos primero el operador de negación (!), reduciendo la expresión a $\text{sunday} \wedge \text{true}$, que se evalúa como true. En la última expresión, ambas variables son true, por lo que se reduce a $\neg(\text{true} \wedge \text{true})$, que a su vez se reduce a $\neg \text{true}$, es decir, false. Por estas razones, la opción F es correcta.

14. Which of the following statements are correct? (Choose all that apply.)
- A. The return value of an assignment operation expression can be void.
 - B. The inequality operator (!=) can be used to compare objects.
 - C. The equality operator (==) can be used to compare a boolean value with a numeric value.
 - D. During runtime, the & and | operators may cause only the left side of the expression to be evaluated.
 - E. The return value of an assignment operation expression is the value of the newly assigned variable.
 - F. In Java, 0 and false may be used interchangeably.
 - G. The logical complement operator (!) cannot be used to flip numeric values.

Mi justificación
X A es false, no creo que sea posible regresar void V B es true ya que por definición es un operador de comparación X C es false ya que no puedes comparar booleanos con numéricos X D es false ya que eso ocurre con &&, no con & V E es true por definición X F es false por lo mismo que la C V G es true ya que el signo de admiración es solo para booleanos.

14. B, E, G. El valor que devuelve una operación de asignación dentro de una expresión es el mismo valor que se asignó a la variable. Por esta razón, la opción A es incorrecta y E es correcta. La opción B es correcta porque los operadores de igualdad (==) y desigualdad (!=) pueden utilizarse con objetos. La opción C es incorrecta, ya que los tipos boolean y numéricos no son comparables. Por ejemplo, no se puede escribir true == 3 porque produciría un error de compilación. La opción D es incorrecta porque los operadores lógicos evalúan ambos lados de la expresión. El operador | hará que ambos lados sean evaluados. La opción F es incorrecta porque Java no acepta números como valores boolean. Finalmente, la opción G es correcta porque se necesita utilizar el operador de negación (-) para cambiar el signo de valores numéricos, no el operador de complemento lógico (!).

15. Which operators take three operands or values? (Choose all that apply.)

A. =

B. &&

C. *=

D. ? :

E. &

F. ++

G. /

Mi justificación
Solo la D ya que requiere de una condición y los outputs sí es true o si es false

15. D. El operador ternario es el único operador que utiliza tres valores, por lo que la opción D es la única correcta. Las opciones A, B, C, E y G son operadores binarios. Aunque pueden encadenarse en expresiones más largas, cada operación utiliza únicamente dos valores. La opción F es un operador unario y utiliza un solo valor.

16. How many lines of the following code contain compiler errors?

```
int note = 1 * 2 + (long)3;  
short melody = (byte)(double)(note *= 2);  
double song = melody;  
float symphony = (float)((song == 1_000f) ? song * 2L : song);
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

Mi justificación

La primera línea marca error de compilación porque $1*2+(long)3$ da como resultado un long, y si queremos asignarlo a una variable int note necesitaríamos un casteo explícito con (int) pero no lo tiene así que es inválido.

La segunda línea no tiene problema ya que $note *= 2$ daría un int, pero tiene casteo (double) y este a su vez tiene casteo (byte) así que el resultado es byte, y si hacemos short Melody = byte, tiene casteo automático short = byte.

La tercera línea de código no tiene problema ya que double = short tiene casteo automático.

La cuarta línea de código no tiene problema ya que $song == 1_000f$ es false, entonces $float\ symphony = (float)(song)$ y tenemos correspondencia de tipos float = float así que todo bien.

Respuesta: B

16. B. La primera línea contiene un error de compilación. El valor 3 se convierte a long. El valor $1 * 2$ se evalúa como int, pero se promociona a long al sumarse con 3. Intentar almacenar un valor long en una variable int provoca un error de compilación. Las demás líneas no contienen errores de compilación, ya que almacenan valores de tipos más pequeños en tipos de datos más grandes o del mismo tamaño; las líneas 2 y 4 utilizan cast. Como solo una línea no compila, la opción B es correcta.

17. Given the following code snippet, what are the values of the variables after it is executed? (Choose all that apply.)

```
int ticketsTaken = 1;
int ticketsSold = 3;
ticketsSold += 1 + ticketsTaken++;
ticketsTaken *= 2;
ticketsSold += (long)1;
```

- A. ticketsSold is 8.
- B. ticketsTaken is 2.
- C. ticketsSold is 6.
- D. ticketsTaken is 6.
- E. ticketsSold is 7.
- F. ticketsTaken is 4.
- G. The code does not compile.

Mi justificación
La última línea genera error de compilación porque no ticketsSold es entero y no le puedes sumar un long a un entero sin hacer casteo explícito. Si quitamos ese (long) entonces ticketsSold = 6 y ticketsTaken = 4. Respuesta: G

17. **C, F.** Los valores iniciales de ticketsTaken y ticketsSold son 1 y 3, respectivamente. Después de la primera asignación compuesta, ticketsTaken aumenta a 2. El valor de ticketsSold aumenta de 3 a 5; como se utilizó el operador de post-incremento, ticketsTaken++ devuelve 1. En la siguiente línea, ticketsTaken se duplica a 4. En la última línea, ticketsSold aumenta en 1 hasta 6. Los valores finales de las variables son 4 y 6 para ticketsTaken y ticketsSold, respectivamente, haciendo que C y F sean correctas. Observa que la última línea no produce un error de compilación porque el operador compuesto realiza automáticamente el cast del operando derecho.

18. Which of the following can be used to change the order of operation in an expression?
(Choose all that apply.)

A. []

B. < >

C. ()

D. \ /

E. { }

F. " "

Mi justificación
Solo los paréntesis pueden cambiar el orden de las operaciones del mismo nivel de importancia como * y %. Los demás son para estructuras como listas, arreglos y el "" para String Respuesta: C

18. C. Solo los paréntesis () pueden utilizarse para cambiar el orden de las operaciones en una expresión, por lo que C es correcta. Los otros símbolos, como [], < > y { }, no pueden utilizarse como paréntesis en Java.

19. What is the result of executing the following code snippet? (Choose all that apply.)

```
3: int start = 7;  
4: int end = 4;  
5: end += ++start;  
6: start = (byte)(Byte.MAX_VALUE + 1);
```

- A. start is 0.
- B. start is -128.
- C. start is 127.
- D. end is 8.
- E. end is 11.
- F. end is 12.
- G. The code does not compile.
- H. The code compiles but throws an exception at runtime.

Mi justificación
El valor máximo de una variable byte es 127, y hasta donde sé, si se le suma 1 regresamos al valor más pequeño de byte, el cual es -128. Entonces start = -128 Para end, tenemos end += ++start, entonces end += 4 + (7 + 1) y así end = 12. Respuesta: B y F

19. B, F. El código compila y se ejecuta correctamente, por lo que G y H son incorrectas. En la línea 5, el operador de pre-incremento se ejecuta primero, por lo que start aumenta a 8 y el nuevo valor se devuelve como el lado derecho de la expresión. El valor de end se calcula sumando 8 al valor original de 4, dejando un nuevo valor de 12 para end, por lo que F es correcta. En la línea 6, estamos incrementando byte más allá de su valor máximo. Debido al overflow, esto produce un número negativo, haciendo que B sea correcta. Incluso si no conocieras el valor máximo de byte, deberías saber que el código compila y se ejecuta, y buscar la respuesta relacionada con start y un número negativo.

20. Which of the following statements about unary operators are true? (Choose all that apply.)
- A. Unary operators are always executed before any surrounding numeric binary or ternary operators.
 - B. The `-` operator can be used to flip a boolean value.
 - C. The pre-increment operator (`++`) returns the value of the variable before the increment is applied.
 - D. The post-decrement operator (`--`) returns the value of the variable before the decrement is applied.
 - E. The `!` operator cannot be used on numeric values.
 - F. None of the above

Mi justificación
X A es false ya que las operaciones unitarias post se ejecutan al final de todo X B es false ya que <code>-</code> solo funciona como operador numérico X C es false ya que el prefijo pre indica que el incremento se realiza antes que el return V D es true ya que el prefijo post indica que primero se realiza el return y luego el decremento V E es true ya que el signo de admiración es operador exclusivamente lógico, no numérico. X F es false ya que D y E son true

20. **A, D, E.** Los operadores unarios tienen la mayor precedencia, por lo que A es correcta. El operador de negación (`-`) se utiliza únicamente con valores numéricos, mientras que el operador de complemento lógico (`!`) se utiliza exclusivamente con valores boolean. Por estas razones, B es incorrecta y E es correcta. Finalmente, los operadores de pre-incremento/pre-decremento devuelven el nuevo valor de la variable, mientras que los operadores de post-incremento/post-decremento devuelven el valor original de la variable. Por estas razones, C es incorrecta y D es correcta.

21. What is the result of executing the following code snippet?

```
int myFavoriteNumber = 8;
int bird = ~myFavoriteNumber;
int plane = -myFavoriteNumber;
var superman = bird == plane ? 5 : 10;
System.out.println(bird + "," + plane + "," + --superman);
```

- A. -7,-8,9
- B. -7,-8,10
- C. -8,-8,4
- D. -8,-8,5
- E. -9,-8,9**
- F. -9,-8,10
- G. None of the above

Mi justificación
Si myFavoriteNumber = 8, bird = -(myFavoriteNumber + 1) según la definición del operando ~ Entonces bird = -9, La variable plane no es más que el negativo de myFavoriteNumber así que plane = -8 La variable superman en la línea 4 vale 10 porque bird == plane es false así que se asigna 10, y al ser imprimido con --superman, se resta 1 antes de imprimirse, entonces superman = 9. Así, queda -9, -8 y 9. Respuesta: E

21. E. El complemento a nivel de bits de 8 se puede encontrar multiplicando el número por -1 y restando 1, haciendo que bird tenga el valor -9. En cambio, plane es -8 porque se niega myFavoriteNumber. Como bird y plane no son iguales, superman recibe el valor 10. El operador de pre-decremento toma superman, le resta 1 y devuelve el nuevo valor, mostrando 9. Por esta razón, la opción E es correcta.

Resultado final: 16/21

Calificación = 7.6